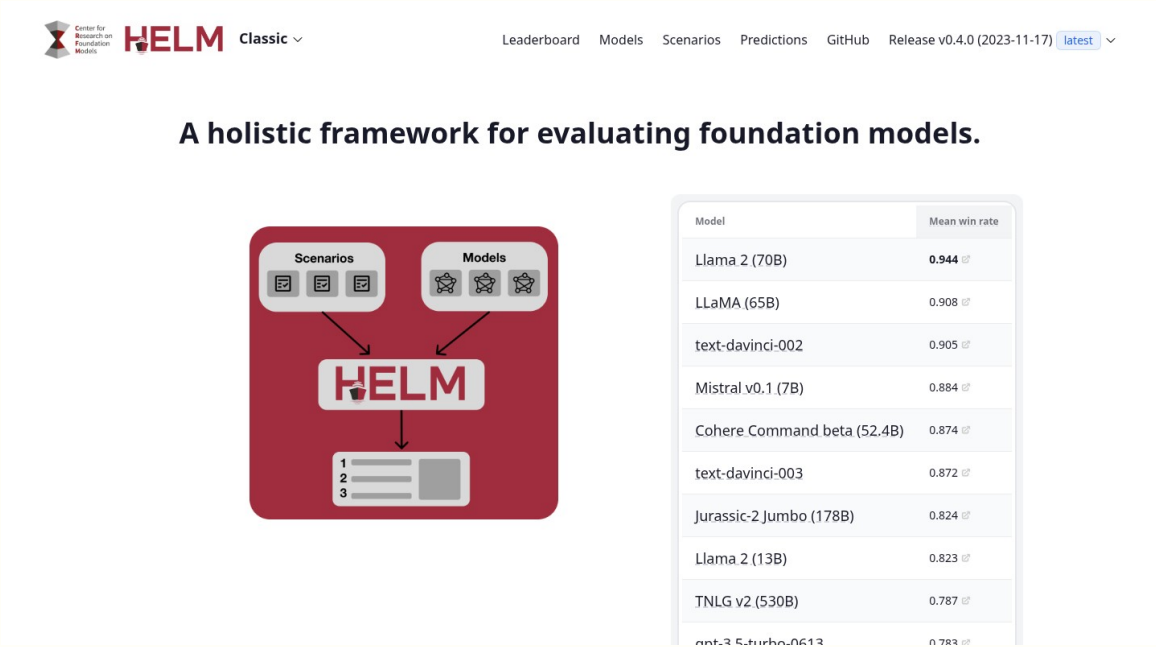
Numeracy

HELM の多項式帰納シナリオ

入出力の例から数の規則を推測する

sasakiy84

公式サイト： HELM Classic



[1] Stanford CRFM. HELM Classic v0.4.0 (2023-11-17). https://crfm.stanford.edu/helm/classic/v0.4.0/

1. 入出力の例から規則を推測する
2. 多項式と入力点を生成する
3. 数値の採点と公開実行
4. 再現方法と利用の範囲

1. 入出力の例から規則を推測する

式を示さず、数の対応を示す

- Numeracy は、 HELM で導入された人工生成の数値推論課題。入力と出力の組から式を推測する。
- モデルには数値の組を示し、新しい入力に対応する値を出力させる。式そのものは入力に書かない。
- ここでは、公開コード v0.3.0 の生成器を seed=0 で実行し、生成された実際のレコードを使う。

[1] Liang et al. (2023). Holistic Evaluation of Language Models. TMLR. https://arxiv.org/abs/2211.09110
[2] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

訓練レコードをプロンプトの例に使う

- 数値の対応を見せる train と、予測対象にする test を生成する。関数を推測するための例をプロンプトに示す。
- standard ／ example の設定では、訓練例は最大 100 件、評価は 100 件、 1 試行。
- 以下のページには、訓練 100 件の連続した冒頭 4 〜 6 件と、 test 入力 1 件の全文を示す。

[1] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py
[2] Stanford CRFM. get_numeracy_spec, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/run_specs.py

実例 1 ：一次式の未知の値を求める

linear / mode=standard / seed=0 / test[0]

原データ：訓練 100 件の先頭 4 件（連続）

```
x, y
-82, -324
70, 284
7, 32
-2, -4
```

予測対象： test レコードの入力（全文）

```
15
```

正解： 64

x=15 のときの y を予測する。

参照メタデータ（入力には含めない）

y = 4x + 4

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

一次式では変化の割合が一定になる

- この生成例の参照式は y=4x+4 。 x=15 なら、 4×15+4=64 になる。
- 入力として渡すのは、−82→−324 、 70→284 などの値の組で、参照式は採点・検証用のメタデータに保存される。
- 数字の並びから規則を推測し、新しい点へ適用することを測る。

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

実例２：二次式の増え方を推測する

parabola / mode=standard / seed=0 / test[0]

原データ：訓練 100 件の先頭 4 件（連続）

```
x, y
18, 667
170, 57971
107, 23006
98, 19307
```

予測対象： test レコードの入力（全文）

```
115
```

正解： 26566

x=115 のときの y を予測する。

参照メタデータ（入力には含めない）

y = 2x^2 + x + 1

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

二次式では入力の大きさが強く効く

- この例の参照式は $y=2x^2+x+1$ 。 $x=115$ なら 26,566 になる。
- 訓練用の $x=107$ に対する値は 23,006 。入力が 8 増えると出力は 3,560 増え、一次式の一定の増え方とは異なる。
- 式の形や係数を入力に直接示さず、数値の対応から推測させる。

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

実例３：二つの入力から出力を求める

plane / mode=standard / seed=0 / test[0]

原データ：訓練 100 件の先頭 6 件（連続）

```
x, y, z
6, 8, 57
-2, 1, -3
0, 6, 25
2, 3, 21
-3, 7, 17
-3, 1, -7
```

予測対象： test レコードの入力（全文）

```
6, 7
```

正解： 53

x=6 、 y=7 のときの z を予測する。

参照メタデータ（入力には含めない）

z = 4x + 4y + 1

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

入力の変数を二つに増やす

- この例は z=4x+4y+1 。 x=6 、 y=7 を代入すると 53 になる。
- 入力の文字列は「 6, 7 」、正解の文字列は「 53 」。モデルは出力する数値だけを生成する。
- 二つの変数を持つ一次式と、二次式までの 4 種類を用意している。

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

2. 多項式と入力点を生成する

4 種類の式で次数と変数の数を変える

コードの型名	変数の数	次数	固定された参照式
linear	1	1	$y = 2x + 5$
parabola	1	2	$y = x^2 + 2$
plane	2	1	$z = 2x + y + 5$
paraboloid	2	2	$z = 2x^2 + y^2 + 2$

右列は example モードの式。 standard モードでは係数を範囲内で抽選する。

[1] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

整数係数と整数の入力点を抽選する

- 各関数型に係数の範囲を決め、 Python と NumPy の乱数を同じ seed で初期化する。
- 最高次数、変数の数、非ゼロの定数項を満たす式ができるまで抽選する。
- 生成した式に入力点を代入し、出力の整数値と、参照式・関数型をレコードへ保存する。

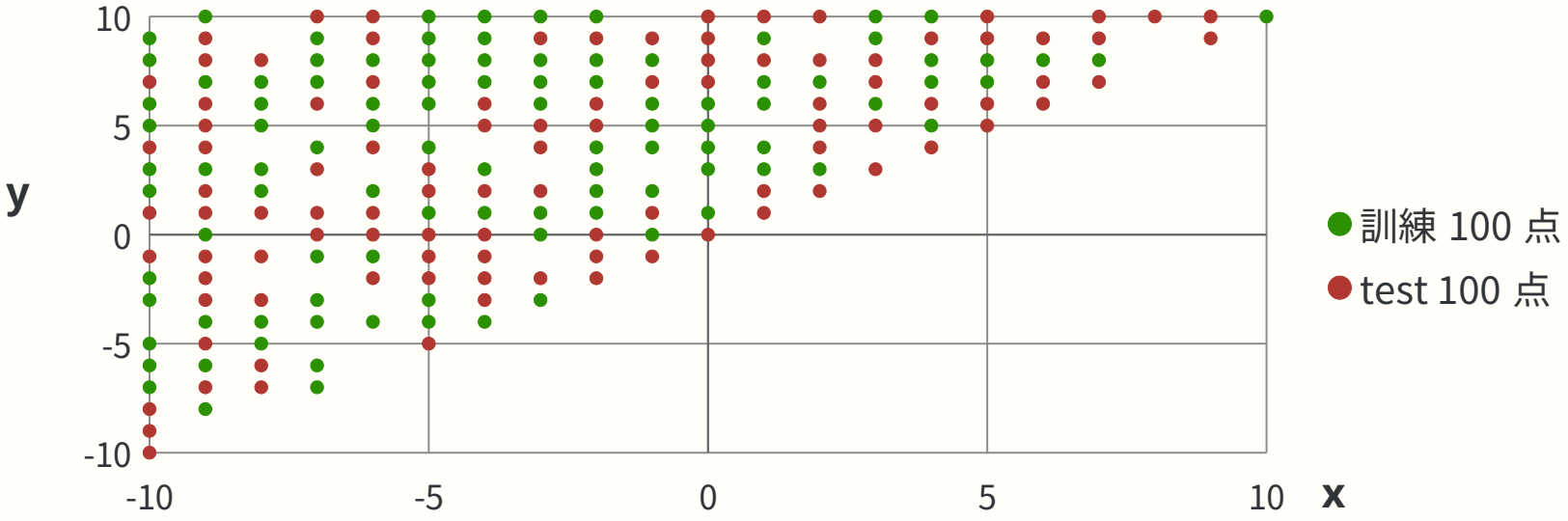
[1] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

入力範囲は関数型に合わせて変える

- linear は−100 〜 100 、 parabola は 0 〜 200 の整数を使う。二次式の入力は非負にしている。
- plane は x ・ y が−10 〜 10 、 paraboloid は 0 〜 20 。さらに 2 変数では x≤y の点だけを残す。
- overlap=True は訓練と test の入力範囲を共通にする設定。訓練用と評価用の点は重複しない。

[1] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

2 変数の入力は三角形の範囲にある



plane ／ standard ／ seed=0 。訓練 100 点、 test 100 点。候補は 231 点。

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

100 点を見せ、別の 100 点で確かめる

- 各関数について、訓練用と test 用に異なる 100 点ずつを選ぶ。 1 変数では 201 候補点、 2 変数では $x \le y$ を満たす 231 候補点がある。
- 今回は各型で seed=0 を使い、 example と standard の 8 組、計 1,600 レコードを生成した。
- 全 1,600 件の正解を参照式へ別途代入して再計算し、生成器の出力と一致することを確認した。

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

モードごとに式の扱いを変える

モード	式の選び方	v0.3.0 の生成件数
example	型ごとに固定された式	train 100 ＋ test 100
standard	1 個の式を抽選する	train 100 ＋ test 100
function	問題ごとに式を替える設計	0 （生成処理は未実装）

各型・ seed=0 を実行して確認。 function 内の生成処理は TODO と return [] が残る。

[1] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

3. 数値の採点と公開実行

正解の文字列と数値のずれを採点する

- 既定では exact_match 、正規化してから比べる quasi_exact_match 、絶対誤差を測る absolute_value_difference を使う。
- exact_match は正解文字列と一致すれば 1 、違えば 0 。absolute_value_difference は、整数として解釈した予測値と正解値の差の絶対値。
- 入力ごとの値を集計する。絶対誤差は小さいほどよく、完全に一致する予測なら 0 になる。

[1] Stanford CRFM. get_numeracy_spec, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/run_specs.py
[2] Stanford CRFM. BasicMetric, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/basic_metrics.py

同じ正解 64 に対する出力の採点

出力	完全一致	正規化後の一致	絶対誤差
64	1	1	0
65	0	0	1
64.0	0	0	64
-64	0	1	128

正規化後の一致は quasi_exact_match 。符号も句読点として除かれ、 -64 と 64 が一致する。

[1] Stanford CRFM. BasicMetric, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/basic_metrics.py

整数として読めない出力の扱い

- absolute_value_difference の実装では、整数として解釈できない出力は、 0 として採点される。
- たとえば正解が 0 のとき、出力が error でも絶対誤差は 0 になる。 exact_match は 0 で、正解文字列とは一致しない。
- 数値のずれと出力形式を合わせて確認できるよう、 exact match と実際の出力を併せて保存する。

[1] Stanford CRFM. BasicMetric, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/basic_metrics.py

曲線・曲面からの距離も定義される

- run_solver=True なら、予測した点から参照曲線・曲面までの最短距離と、整数として読めた出力の割合を追加できる。
- 実例 1 で 65 を予測すると、点 (15, 65) は直線 y=4x+4 から 1/√17 、約 0.243 だけ離れる。
- これは任意の追加指標。既定の指標は数値の絶対誤差であり、 Classic の表示設定でも主指標になっている。

[1] Stanford CRFM. get_numeracy_spec, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/run_specs.py
[2] Stanford CRFM. DistanceMetric, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/numeracy_metrics.py

Classic v0.4.0 の公開実行は 0 件

確認した対象	公開内容
シナリオ定義	numeracy が登録されている
主指標	absolute_value_difference / test
公開 run_specs	該当 run は 0 件
モデル別の成績	掲載された値はない

2026-10-18 に latest 設定と v0.4.0 の配布 JSON を確認した。

[1] Stanford CRFM. HELM Classic v0.4.0 (2023-11-17). https://crfm.stanford.edu/helm/classic/v0.4.0/

生成できる設定と公開成績を区別する

- 公開 run は 0 件だった。 0 点というモデル成績が記録されているのではない。
- v0.3.0 の example ／ standard はレコードを生成する一方、既定の function は空のリストを返す。
- この資料では、動作する生成器の出力と採点実装を説明する。未実行のモデル得点を補っていない。

[1] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py
[2] Stanford CRFM. HELM Classic v0.4.0 (2023-11-17). https://crfm.stanford.edu/helm/classic/v0.4.0/

4. 再現方法と利用の範囲

規則の推測と適用を切り分けて調べる

- 一つの式に多くの入出力例を与え、未知の入力点へ適用できるかを調べる。
- 入力範囲、係数範囲、訓練例の数、表示順序を固定すると、数値の規則への適合を比較する評価を組める。
- 答えの規模によって、同じ割合のずれでも絶対誤差が変わる。関数型と出力範囲も結果と対応させる。

[1] Liang et al. (2023). Holistic Evaluation of Language Models. TMLR. https://arxiv.org/abs/2211.09110
[2] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

公式生成器でレコードを作る

```
scenario = NumeracyScenario(
    relation_type="linear",
    mode="standard", seed=0
)
records = scenario.get_instances("output")
len(records)  # 200
```

HELM v0.3.0 の NumeracyScenario を読み込んだ後の例。 train 100 件と test 100 件が返る。

[1] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

再現にはバージョンと乱数を保存する

- 再現には seed に加えて実行環境も保存する。ここでは Python 3.12 、 NumPy 2.3.5 、 SymPy 1.14.0 で公式の生成処理を実行した。
- seed 、関数型、モードに加え、 overlap=True 、 sort_vals=False を固定した。
- 公式ソースの SHA-256 、全生成レコード、 13 実例の位置とハッシュを保存した。

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

公開コードを使って人工データを生成する

- HELM のコードは Apache License 2.0 で公開されている。Numeracy の入力と正解は、このコードが人工的に生成する。
- 人が書いた問題文や既存 Web 文章のコレクションをダウンロードする課題ではない。
- 公式リポジトリの v0.3.0 を選び、生成器と採点実装を同じバージョンでそろえて使う。

[1] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py
[2] Stanford CRFM. HELM v0.3.0, Apache License 2.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/LICENSE

数値の対応から未知の値を予測する

- Numeracy は、一次・二次、 1 変数・ 2 変数の多項式を使い、例から規則を推測して新しい入力へ適用する課題。
- 既定の採点には文字列一致と絶対誤差があり、整数として読めない出力の扱いも得点に影響する。
- Classic v0.4.0 には公開モデル成績がない。動作する example ／ standard の生成器から、条件を固定して評価を組み立てられる。

[1] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py
[2] Stanford CRFM. HELM Classic v0.4.0 (2023-11-17). https://crfm.stanford.edu/helm/classic/v0.4.0/

Appendix

1. 追加の実例
2. 生成と採点の詳細
3. 研究と参考文献

実例 4 ：交差項を含む二次式

paraboloid / mode=standard / seed=0 / test[0]

原データ：訓練 100 件の先頭 6 件（連続）

```
x, y, z
16, 18, 1128
8, 11, 341
10, 16, 620
12, 13, 617
7, 17, 510
7, 11, 300
```

予測対象： test レコードの入力（全文）

```
16, 17
```

正解： 1077

x=16 、 y=17 のときの z を予測する。

参照メタデータ（入力には含めない）

z = 2x^2 + xy + y^2 + 4

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

実例 5 ：固定された一次式

linear / mode=example / seed=0 / test[0]

原データ：訓練 100 件の先頭 4 件（連続）

```
x, y
-82, -159
70, 145
7, 19
-2, 1
```

予測対象： test レコードの入力（全文）

```
15
```

正解： 35

x=15 のときの y を予測する。

参照メタデータ（入力には含めない）

y = 2x + 5

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

実例 6 ：固定された二次式

parabola / mode=example / seed=0 / test[0]

原データ：訓練 100 件の先頭 4 件（連続）

```
x, y
18, 326
170, 28902
107, 11451
98, 9606
```

予測対象： test レコードの入力（全文）

```
115
```

正解： 13227

x=115 のときの y を予測する。

参照メタデータ（入力には含めない）

y = x^2 + 2

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

実例 7 ：固定された平面の式

plane / mode=example / seed=0 / test[0]

原データ：訓練 100 件の先頭 6 件（連続）

```
x, y, z
6, 8, 25
-2, 1, 2
0, 6, 11
2, 3, 12
-3, 7, 6
-3, 1, 0
```

予測対象： test レコードの入力（全文）

```
6, 7
```

正解： 24

x=6 、 y=7 のときの z を予測する。

参照メタデータ（入力には含めない）

z = 2x + y + 5

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

実例 8 ：固定された二次曲面

paraboloid / mode=example / seed=0 / test[0]

原データ：訓練 100 件の先頭 6 件（連続）

```
x, y, z
16, 18, 838
8, 11, 251
10, 16, 458
12, 13, 459
7, 17, 389
7, 11, 221
```

予測対象： test レコードの入力（全文）

```
16, 17
```

正解： 803

x=16 、 y=17 のときの z を予測する。

参照メタデータ（入力には含めない）

z = 2x^2 + y^2 + 2

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

実例９：正解が 0 になる入力

linear / mode=standard / seed=0 / test[1]

原データ：訓練 100 件の先頭 4 件（連続）

```
x, y
-82, -324
70, 284
7, 32
-2, -4
```

予測対象： test レコードの入力（全文）

```
-1
```

正解： 0

x=-1 のときの y を予測する。

参照メタデータ（入力には含めない）

y = 4x + 4

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

実例 10 ：二次式の隣の入力点

parabola / mode=standard / seed=0 / test[1]

原データ：訓練 100 件の先頭 4 件（連続）

```
x, y
18, 667
170, 57971
107, 23006
98, 19307
```

予測対象： test レコードの入力（全文）

```
99
```

正解： 19702

x=99 のときの y を予測する。

参照メタデータ（入力には含めない）

y = 2x^2 + x + 1

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

実例 11 ：二つの入力がともに大きい

plane / mode=standard / seed=0 / test[1]

原データ：訓練 100 件の先頭 6 件（連続）

```
x, y, z
6, 8, 57
-2, 1, -3
0, 6, 25
2, 3, 21
-3, 7, 17
-3, 1, -7
```

予測対象： test レコードの入力（全文）

```
7, 10
```

正解： 69

x=7 、 y=10 のときの z を予測する。

参照メタデータ（入力には含めない）

z = 4x + 4y + 1

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

実例 12 ：二次曲面の別の入力点

paraboloid / mode=standard / seed=0 / test[1]

原データ：訓練 100 件の先頭 6 件（連続）

```
x, y, z
16, 18, 1128
8, 11, 341
10, 16, 620
12, 13, 617
7, 17, 510
7, 11, 300
```

予測対象： test レコードの入力（全文）

```
17, 20
```

正解： 1322

x=17 、 y=20 のときの z を予測する。

参照メタデータ（入力には含めない）

z = 2x^2 + xy + y^2 + 4

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

実例 13 ：一次式の近い入力点

linear / mode=standard / seed=0 / test[2]

原データ：訓練 100 件の先頭 4 件（連続）

```
x, y
-82, -324
70, 284
7, 32
-2, -4
```

予測対象： test レコードの入力（全文）

```
14
```

正解： 60

x=14 のときの y を予測する。

参照メタデータ（入力には含めない）

y = 4x + 4

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

standard モードの係数範囲

型	式と係数（両端を含む整数）
linear	$y=Ax+B$ ： A, B は 1 〜 5
parabola	$y=Ax^2+Bx+C$ ： A は 1 〜 2 、 B は 0 〜 2 、 C は 1 〜 5
plane	$z=Ax+By+C$ ： A, B, C は 1 〜 5
paraboloid	$z=Ax^2+Bxy+Cy^2+F$ A, C は 1 〜 2 、 B は 0 〜 1 、 F は 1 〜 5

[1] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

2 変数の二次式には交差項も現れる

- v0.3.0 の係数範囲では、 paraboloid の xy の係数は 0 または 1 。 x と y に関する一次項は 0 に固定される。
- standard ／ seed=0 の参照式は $z=2x^2+xy+y^2+4$ となり、交差項 xy を含む。
- コードのコメントには axis-aligned のみとあるが、実際の係数範囲と生成結果には交差項がある。

[1] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

seed=0 の standard 出力の範囲

型	train + test の最小値	最大値
linear	-396	404
parabola	1	80,201
plane	-79	81
paraboloid	4	1,604

各型の 200 件を実測した範囲。係数と入力点の全組み合わせの極値ではない。

[1] HELM v0.3.0 の公式生成器を seed=0 で実行。 Python 3.12 / NumPy 2.3.5 。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

レコードには正解と参照式を保存する

- references の先頭は数値の正解で、 correct タグが付く。次に solution タグの参照式、 class タグの関数型が入る。
- モデルの入力は input.text にある整数の列。採点と検証に使う参照式は、通常の入出力例の文章に混ぜない。
- 実例の ID は、関数型・モード・ seed ・ test 配列内の位置を組み合わせた、この資料での識別子。

[1] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

function モードは生成処理が止まっている

- function は、問題ごとに式を抽選し直す設計として説明される。 MODE_INFO には訓練・評価の関数を各 1,000 個とする設定がある。
- しかし v0.3.0 の generate_datasets は TODO とともに空リストを返す。 4 型を seed=0 で実行して、いずれも 0 件だった。
- 設定上の 1,000 関数と、実際に生成・公開評価された件数を同じ数量として扱わない。

[1] Stanford CRFM. NumeracyScenario, HELM v0.3.0. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

Numeracy-600K は別の自然言語データ

- Numeracy-600K は、ニュース見出しや市況コメントに現れる数値の大きさを扱う、 2019 年の別データセット。
- HELM の Numeracy は、人工生成した入出力の組から多項式の値を推測する。質問・文章や利用する指標が異なる。
- HELM の論文では数値関係を推測する課題を numeracy_prediction と呼び、公開コードでは numeracy として実装している。

[1] Chen, Huang, Takamura & Chen (2019). Numeracy-600K. ACL. https://aclanthology.org/P19-1635/
[2] Liang et al. (2023). Holistic Evaluation of Language Models. TMLR. https://arxiv.org/abs/2211.09110

参考文献

Holistic Evaluation of Language Models

Percy Liang et al. (2023). Transactions on Machine Learning Research.

幅広いシナリオと指標で言語モデルを評価する枠組みと、数値関係の帰納を含む推論課題を示す。

https://arxiv.org/abs/2211.09110

NumeracyScenario

Stanford CRFM (2023). HELM v0.3.0, Python implementation.

多項式の係数・入力点の生成、 train ／ test の構成、参照式と正解の記録を実装する。

https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/numeracy_scenario.py

参考文献

get_numeracy_spec

Stanford CRFM (2023). HELM v0.3.0, run_specs.py.

モード別の提示例数、評価件数、生成方法と既定・任意の指標を設定する。

https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/run_specs.py

BasicMetric: absolute_value_difference

Stanford CRFM (2023). HELM v0.3.0, basic_metrics.py.

出力の整数変換と絶対誤差、文字列一致などの基本的な採点処理を実装する。

https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/basic_metrics.py

参考文献

DistanceMetric

Stanford CRFM (2023). HELM v0.3.0, numeracy_metrics.py.

予測点と曲線・曲面の最短距離、および整数として読める出力の割合を計算する。

https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/numeracy_metrics.py

Numeracy-600K: Learning Numeracy for Detecting Exaggerated Information in Market Comments

Chung-Chi Chen, Hen-Hsen Huang, Hiroya Takamura, Hsin-Hsi Chen (2019). ACL.

ニュース見出しと市況コメントの数値を集め、数の大きさの理解と誇張表現の検出を扱う。

https://aclanthology.org/P19-1635/